

1-1 In a Turnpike Reconstruction Problem, given distance set $D = 1, 2, 3, 4, 5, 6$, $(x_1, \dots, x_4) = (0, 2, 5, 6)$ is not the only solution provided that $x_1 = 0$.

1-2 After merging two Leftist Heaps H_1 and H_2 of different NPL's, the NPL of the resulted Leftist Heap's root could not be $\max(NPL(H_1), NPL(H_2)) + 1$.

1-3 Finding the minimum key from a splay tree will result in a tree with its root having no right subtree.

(F)

1-4 To solve a problem by dynamic programming instead of recursion, the key approach is to avoid storing the results of computations for the subproblems, and instead recompute them every time they are needed, even if the same subproblem must be computed repeatedly.

1-5 After inserting $\{6, 3, 1, 2, 4, 5, 9, 8, 7, 10, 11, 12, 13\}$ to a 3-order B+ tree, the sum of values in the root is 14.

(F)

1-6 An AVL tree with 20 nodes must have a height of 5 (where a root-only tree has height 0).

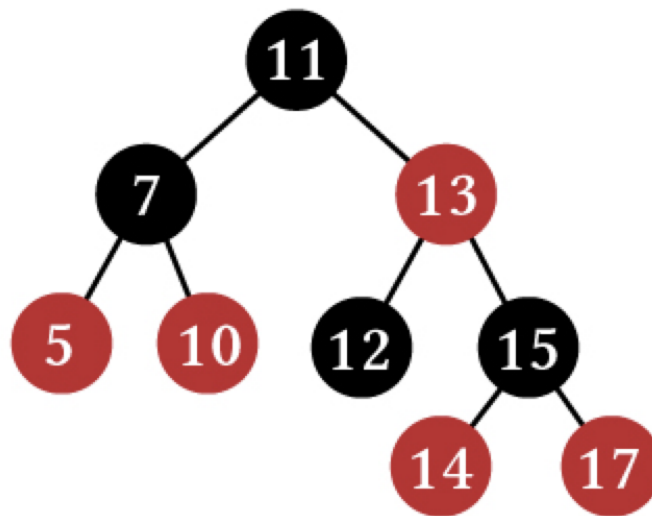
(F)

1-7 Compared to Binary Heap, Binomial Heap provides a better $O(\log N)$ amortized 'Merge' bound.

1-8 The recurrence $T(n) = 2T(n/2) + n/\log n$ can be solved by the Master Theorem introduced in our class.

1-9 In a distributed index, document-partitioning requires a single-term query (e.g., 'algorithm') to be sent to all shards, whereas term-partitioning can route the query to a single shard.

1-10 After deleting 12 from the following red-black tree, 13, 14, 15 and 17 will change its color.



2-1

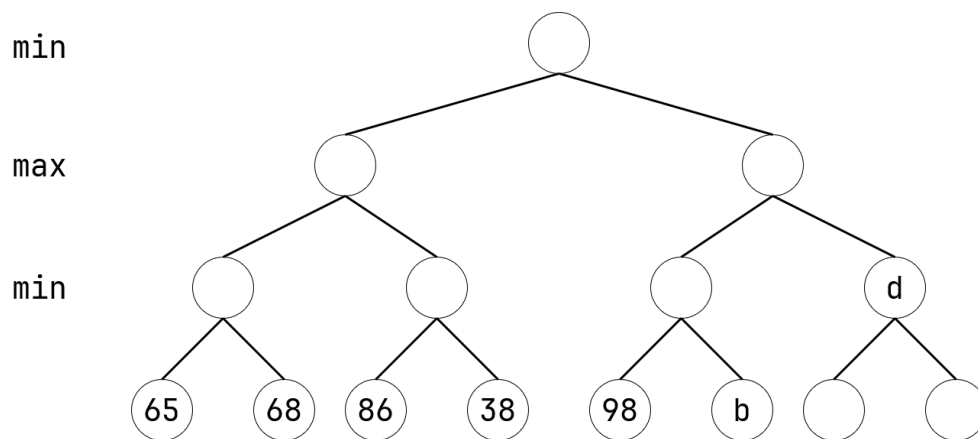
Insert {3,2,9,0,4,5,6,1,7} into an initially empty 2-3 B tree (with splitting). Which one of the following statements is FALSE?

2-2

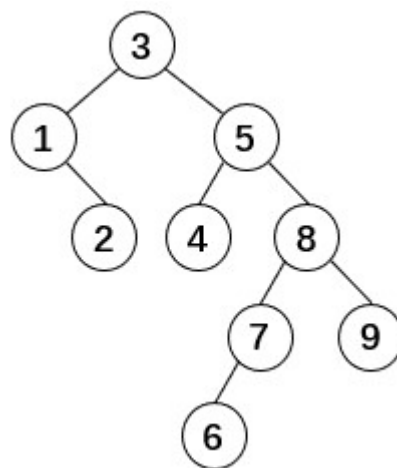
Delete the minimum number from the given binomial queue in the following figure. Which one of the following statements is FALSE?

- A. 11 is the root of a binomial tree
- B. 29 and 23 are both children of 4
- C. there are three binomial trees after deletion, which are B_0 , B_1 and B_4
- D. 23 is not the root of any resulting binomial tree

2-3 Given the following game tree, node d will be pruned with $\alpha-\beta$ pruning algorithm if and only if



2-4 After deleting 7 from the given splay tree, which of the following statements about the resulting tree is impossible?



- A.1 and 4 are siblings
- B.6 is the grandparent of 5
- C.8 is the root
- D.6 is the root

2-5 In the context of Red-Black Tree, how many of the following statements are **True**?

- In the worst case the DELETE operation in a Red-Black tree of nodes requires $\Omega(\log^{**}n)$ rotations.
- In a Red-Black tree with 3 internal nodes, there must be a red internal node.

- When inserting a node into a red-black tree, we shall first insert the node as into an ordinary binary search tree, and then color the node black.
- In a Red-Black tree, the path from the root to the nearest leaf is no more than half as long as the path from the root to the farthest leaf.

A. 3 B. 1 C. 2 D. 0

2-6 A college student is preparing for their midterm exams. They have a list of n topics to study. For each topic i , they have an associated `expected_hours[i]` to master it and an `expected_grade_increase[i]` they will receive upon mastering it. They have a total of `H` hours to study. The student wants to maximize their total grade increase. However, there's a catch: for every topic i they master, they also experience "fatigue," which adds an additional `fatigue_factor[i]` hours to the study time of *all subsequent* topics they choose to study.

For example, if they decide to study topic A first, with `expected_hours[A]`, and then topic B, the time taken for topic B will be `expected_hours[B] + fatigue_factor[A]`. If they then study topic C, the time for C will be `expected_hours[C] + fatigue_factor[A] + fatigue_factor[B]`. The order in which they study the topics matters.

Which of the following dynamic programming recurrence relations correctly computes the maximum grade increase the student can achieve?

Let `dp(i, h, f)` be the maximum grade increase achievable by considering topics from index i to $n-1$, with h hours remaining and a total accumulated fatigue f from previously studied topics.

A.

```
dp(i, h, f) = max(dp(i+1, h, f),
expected_grade_increase[i] + dp(i, h - expected_hours[i]
- f, f + fatigue_factor[i]))
```

B.

```
dp(i, h, f) = max(dp(i+1, h, f),
expected_grade_increase[i] + dp(i+1, h -
expected_hours[i] - f, f + fatigue_factor[i]))` if `h >=
expected_hours[i] + f
```

C.

This problem cannot be optimally solved with dynamic programming due to the ordering dependency of fatigue. It requires a greedy approach.

D.

```
dp(i, h) = max(dp(i-1, h), expected_grade_increase[i] +
dp(i-1, h - expected_hours[i]))
```

2-7 You are tasked with analyzing a dynamic array data structure that supports `push` and `pop` operations. The array has a capacity `m` and contains `n` elements. The resizing logic is as follows:

- **`push(x)`** : If the array is full (`n == m`), a new array of capacity `2m` is allocated, all `n` elements are copied, and then `x` is added. The actual cost of this operation is `n + 1` . If the array is not full (`n < m`), the element is added in constant time, for an actual cost of `1` .
- **`pop()`** : An element is removed. If this removal causes the number of elements to become less than or equal to one-quarter of the capacity (i.e., `n-1 ≤ m/4`), a new array of capacity `m/2` is allocated and the `n-1` elements are copied. The actual

cost of this operation is $(n-1) + 1 = n$. If the array does not need to shrink, the actual cost is 1 .

Your goal is to find a potential function Φ , expressed in terms of n and m , that proves the amortized cost of both `push` and `pop` is $O(1)$. The potential Φ must always be non-negative.

Which of the following potential functions correctly achieves this?

- A. $\Phi(n,m) = 2n - m$
- B. $\Phi(n,m) = |n - m/2|$
- C. $\Phi(n,m) = \{2n - m, \text{if } n/m \geq 1/2; m/2 - n, \text{if } n/m < 1/2\}$
- D. $\Phi(n,m) = |2n - m|$

2-8 What is the defining mechanical difference between the merge operation in a Skew Heap and in a Leftist Heap?

- A. Skew Heaps merge along the left spines, while Leftist Heaps merge along the right spines.
- B. Skew Heaps unconditionally swap the left and right children of every node on the merge path, while Leftist Heaps only swap if the NPL property is violated.
- C. Skew Heaps have an $O(\log n)$ worst-case merge time, while Leftist Heaps have an $O(\log n)$ amortized merge time.
- D. Skew Heaps do not need to maintain the min-heap property during the merge, only at the end.

2-9 Which of the following asymptotic upper bounds is correct?

- A. $T(n) = 2T(n-1) + 1$. Then $T(n) = O(n^2)$.
- B. $T(n) = T(n/5) + T(7n/10) + n$. Then $T(n) = O(n \log n)$.
- C. $T(n) = 4T(n/2) + n^2/\log n$. Then $T(n) = O(n^2)$.
- D. $T(n) = 3T(n/3) + n/\log n$. Then $T(n) = O(n \log \log n)$.

Given an integer array `nums`, you need to find the **length** of the longest strictly increasing **subsequence** in `nums`.

A **subsequence** is a sequence derived from the array by deleting some or no elements without changing the order of the remaining elements. For example, `[3, 6, 2, 7]` is a subsequence of the array `[0, 3, 1, 6, 2, 2, 7]`.

We use *dynamic programming* to solve the problem. Please fill up the blanks in the following program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 10000
int max(int a, int b) {
    return (a > b) ? a : b;
}

int lengthOfLIS(int* nums, int numsSize) {
    if (numsSize <= 1) return numsSize;
    int dp[MAX_SIZE];
    for(int i = 0; i < numsSize; i++) {
        dp[i] = 1;
    }

    int result = 1;

    for(int i = 1; i < numsSize; i++) {
        for(_____) {
            if(nums[i] > nums[j]) {
                _____;
            }
            if(dp[i] > result) {
                result = dp[i];
            }
        }
    }
    return result;
}
```

3-2

You are tasked with solving the classic **Eight Queens Problem**, where the goal is to place eight queens on an 8×8 chessboard such that no two queens threaten each other. This means that no two queens share the same **row**, **column**, or **diagonal**.

We will use the *backtracking* method to solve this problem. Please fill in the blanks in the following C functions to complete the algorithm.

```
#include <stdio.h>
#include <stdbool.h>

#define N 8

int solutions = 0;

bool isSafe(int board[N][N], int row, int col) {
    int i, j;
    for(i = 0; i < row; i++) {
        if(board[i][col]) return false;
    }

    for(i = row-1, j = col-1; i >=0 && j >=0; i--, j--)
    {
        if(board[i][j]) return false;
    }

    for(i = row-1, j = col+1; _____) {
        if(board[i][j]) return false;
    }

    return true;
}

void solveNQueens(int board[N][N], int row) {
    if(row == N) {
        solutions++;
    }
    for(int col = 0; col < N; col++) {
        if(isSafe(board, row, col)) {
```



```

        board[row][col] = 1;
        solveNQueens(_____);
        _____;
    }
}
}
int main() {
    int board[N][N];
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            board[i][j] = 0;
        }
    }
    solveNQueens(board, 0);
    printf("%d", solutions);
    return 0;
}

```